Django ModelForm CRUD 가이드

프로젝트 `crud` / 앱 `articles` 기준. 위에서부터 순서대로 작성.

전체 구조

```
00_ModelForm_crud/
└── crud/
    ├── manage.py
    ├── crud/              # 프로젝트 설정
    │   ├── settings.py
    │   └── urls.py
    ├── templates/
    │   └── base.html      # 공통 레이아웃
    └── articles/          # 앱
        ├── models.py      # 테이블 정의
        ├── forms.py       # ModelForm 정의
        ├── urls.py        # 앱 URL
        ├── views.py       # 요청 처리
        └── templates/articles/
            ├── index.html
            ├── create.html
            ├── detail.html
            └── update.html
```

파일	역할
`models.py`	DB 테이블 구조
`forms.py`	모델 기반 입력 양식 + 유효성 검사
`urls.py`	주소 ↔ view 연결
`views.py`	요청 받아 DB 처리 후 응답
`templates/`	화면 HTML

요청 흐름

```
브라우저 → urls.py → views.py → (forms.py / models.py) → templates → 응답
```

---

1. 프로젝트 준비 및 가상환경 설정

bash

```
python -m venv venv
source venv/Scripts/activate      # Windows (git bash)
# source venv/bin/activate        # Mac / Linux

pip install django
django-admin startproject crud .
python manage.py runserver        # http://127.0.0.1:8000 확인
```

명령	의미
`python -m venv venv`	가상환경 생성 (폴더명 `venv`)
`activate`	가상환경 진입 → 프롬프트에 `(venv)` 표시
`startproject crud .`	현재 폴더에 프로젝트 생성 (`.` 주의)

---

2. 앱 생성 및 기본 템플릿 준비

2-1. 앱 생성 + 등록

bash

```
python manage.py startapp articles
```

`crud/settings.py`

python

```
INSTALLED_APPS = [
    'articles',          # 앱 등록 (맨 위)
    'django.contrib.admin',
    ...
]

TEMPLATES = [
    {
        ...
        'DIRS': [BASE_DIR / 'templates'],   # 공통 템플릿 폴더 경로
        'APP_DIRS': True,
        ...
    },
]
```

- `INSTALLED_APPS` : 등록 안 하면 모델·템플릿 인식 불가
- `DIRS` : 앱 밖 `templates/` 폴더를 Django가 찾도록 지정

2-2. 공통 레이아웃

`crud/templates/base.html`

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Document</title>
</head>
<body>
  <header>
    <a href="{% url "articles:index" %}">홈</a> |
    <a href="{% url "articles:create" %}">새 글</a>
  </header>
  {% block content %}{% endblock content %}
</body>
</html>
```

- `{% block content %}` : 각 페이지가 채워 넣을 자리
- 이후 모든 페이지는 `{% extends "base.html" %}` 로 시작

2-3. 모델 + ModelForm

`articles/models.py`

```python
from django.db import models

class Article(models.Model):
    title = models.CharField(max_length=100)
    content = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    def __str__(self):
        return self.title
```

필드	옵션	의미
`created_at`	`auto_now_add=True`	생성 시 1회 자동 저장
`updated_at`	`auto_now=True`	저장할 때마다 갱신

```bash
python manage.py makemigrations   # 설계도 생성
python manage.py migrate          # DB 반영
```

`articles/forms.py`

```python
from django.forms import ModelForm
from .models import Article

class ArticleForm(ModelForm):
    class Meta:
        model = Article
        fields = ('title', 'content',)
        # fields = '__all__'
```

- `model` : 어떤 모델 기준으로 양식 생성할지
- `fields` : 사용자에게 입력받을 필드만 지정 (`created_at` 등 자동 필드 제외)

---

3. URL 작성 (namespace · name · variable routing)

3-1. 프로젝트 URL

`crud/urls.py`

```python
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('articles/', include('articles.urls')),   # articles/ 이하 → 앱 urls.py
]
```

3-2. 앱 URL

`articles/urls.py`

```python
from django.urls import path
from . import views

app_name = 'articles'                                    # namespace

urlpatterns = [
    path('', views.index, name='index'),                 # 목록
    path('create/', views.create, name='create'),        # 생성 화면 + 생성 요청
    path('<int:pk>/', views.detail, name='detail'),      # 상세
    path('<int:pk>/update/', views.update, name='update'),  # 수정 화면 + 수정 요
    path('<int:pk>/delete/', views.delete, name='delete'),  # 삭제 요청
]
```

항목	역할	템플릿에서 사용
`app_name`	URL namespace (앱 구분)	`{% url "articles:index" %}`
`name`	각 URL 별명 → 주소 바뀌어도 템플릿 수정 불필요	`{% url "articles:detail" article.pk %}`
`<int:pk>`	variable routing: 주소 일부를 정수로 받아 view 인자 `pk`로 전달	`articles/3/` → `detail(request, pk=3)`

4. CRUD 작성

기능	URL	method	view	템플릿
index (목록)	`articles/`	GET	`index`	`index.html`
CREATE	`articles/create/`	GET, POST	`create`	`create.html`
READ	`articles/<pk>/`	GET	`detail`	`detail.html`
UPDATE	`articles/<pk>/update/`	GET, POST	`update`	`update.html`
DELETE	`articles/<pk>/delete/`	POST	`delete`	없음 (redirect)

`articles/views.py` 상단 import

```python
from django.shortcuts import render, redirect
from .forms import ArticleForm
from .models import Article
```

4-1. index (목록)

`views.py`

```python
def index(request):
    articles = Article.objects.all().order_by('-pk')
    context = {
        'articles': articles,
    }
    return render(request, 'articles/index.html', context)
```

- `order_by('-pk')` : 최신 글 위로
- `context` : 템플릿에 넘길 데이터 (dict)

`templates/articles/index.html`

```html
```

```
{% extends "base.html" %}
{% block content %}
<h1>게시글 목록</h1>
<div>
  {% for article in articles %}
    <a href="{% url "articles:detail" article.pk %}"><p>{{ article.title }}</p>
  {% endfor %}
</div>
{% endblock content %}
```

- `{% url "articles:detail" article.pk %}` : `<int:pk>` 자리에 `article.pk` 전달

4-2. CREATE

`views.py`

python

```python
def create(request):
    if request.method == 'POST':
        form = ArticleForm(request.POST)        # 사용자 입력 담은 양식
        if form.is_valid():                     # 유효성 검사
            article = form.save()               # DB 저장 + 저장된 객체 반환
            return redirect('articles:detail', article.pk)
    else:
        form = ArticleForm()                    # 빈 양식
    context = {
        'form': form,
    }
    return render(request, 'articles/create.html', context)
```

분기	상황	form
GET	작성 화면 요청	`ArticleForm()` 빈 양식
POST	글쓰기 버튼 클릭	`ArticleForm(request.POST)` 입력값 담긴 양식
POST + 검사 실패	`is_valid()` False	if 밖으로 내려와 에러 포함 양식 다시 렌더

- `redirect('articles:detail', article.pk)` : 저장 후 상세 페이지로 이동 (새로고침 중복 저장 방지)

`templates/articles/create.html`

html

```
{% extends "base.html" %}
{% block content %}
<h1>새 글 작성</h1>
<form action="" method="POST">
  {% csrf_token %}
  {{ form.as_p }}
  <input type="submit" value="글쓰기">
</form>
{% endblock content %}
```

항목	의미
`action=""`	현재 주소(`create/`)로 다시 요청
`method="POST"`	데이터 전송 → view의 POST 분기 실행
`{% csrf_token %}`	POST 요청 필수 보안 토큰 (없으면 403)
`{{ form.as_p }}`	필드마다 `<p>` 로 감싸 출력

4-3. READ (상세)

`views.py`

python

```
def detail(request, pk):
    article = Article.objects.get(pk=pk)
    context = {
        'article': article,
    }
    return render(request, 'articles/detail.html', context)
```

- `pk` : URL `<int:pk>` 에서 전달된 값
- `.get(pk=pk)` : 해당 글 1개 조회

`templates/articles/detail.html`

html

```
{% extends "base.html" %}
{% block content %}
<h1>게시글 상세</h1>
<hr>
<h1>{{ article.title }}</h1>
<hr>
<p>{{ article.content }}</p>
<hr>
<a href="{% url "articles:index" %}">[목록으로]</a>
<a href="{% url "articles:update" article.pk %}">[수정하기]</a>
<form action="{% url "articles:delete" article.pk %}" method="POST">
  {% csrf_token %}
  <input type="submit" value="삭제하기">
</form>
{% endblock content %}
```

- 수정 : `<a>` 링크 (GET → 수정 화면)
- 삭제 : `<form method="POST">` (데이터 변경이므로 GET 링크 금지)

4-4. UPDATE

`views.py`

python

```python
def update(request, pk):
    article = Article.objects.get(pk=pk)
    if request.method == 'POST':
        form = ArticleForm(data=request.POST, instance=article)  # 입력값 + 수정
        if form.is_valid():
            form.save()
            return redirect('articles:detail', pk)
    else:
        form = ArticleForm(instance=article)                     # 기존 내용 채워
    context = {
        'form': form,
    }
    return render(request, 'articles/update.html', context)
```

CREATE와 차이	의미
`instance=article`	새 글 생성이 아닌 기존 글 수정
GET에서도 `instance` 전달	양식에 원래 내용 미리 채움

`templates/articles/update.html`

```html
{% extends "base.html" %}
{% block content %}
<h1>게시글 수정</h1>
<form action="" method="POST">
  {% csrf_token %}
  {{ form }}
  <input type="submit" value="수정">
</form>
{% endblock content %}
```

- 구조는 `create.html`과 동일, 제목·버튼 문구만 변경

4-5. DELETE

`views.py`

```python
def delete(request, pk):
    if request.method == 'POST':
        article = Article.objects.get(pk=pk)
        article.delete()
    return redirect('articles:index')
```

- POST일 때만 삭제 → 주소창 직접 입력(GET)으로는 삭제 불가
- 템플릿 없음 : 삭제 버튼은 `detail.html`에 위치, 처리 후 목록으로 redirect

---

5. 실행 확인

```bash
python manage.py runserver
```

#	확인	주소
1	목록 화면 표시	`http://127.0.0.1:8000/articles/`
2	새 글 작성 → 상세로 이동	`/articles/create/`
3	상세 화면 → 수정 → 반영	`/articles/1/update/`
4	삭제 → 목록으로 이동	상세 화면 삭제 버튼

예정된 막힘

증상	원인	대처
`TemplateDoesNotExist: base.html`	`settings.py` `DIRS` 미설정	`'DIRS': [BASE_DIR / 'templates']`
`NoReverseMatch: 'articles' is not a registered namespace`	`app_name` 누락	`articles/urls.py`에 `app_name = 'articles'`
`403 Forbidden (CSRF)`	`{% csrf_token %}` 누락	`<form>` 안에 추가
`no such table: articles_article`	migrate 안 함	`makemigrations` → `migrate`
수정 화면이 빈 양식	GET 분기에 `instance` 누락	`ArticleForm(instance=article)`
`ModuleNotFoundError: django`	가상환경 미활성화	`source venv/Scripts/activate`